

Smart Academic Advisor - Key Code Snippets

Grad project discussion PDF: only the highest-value snippets you are most likely to be asked about: authentication, role protection, data loading, advisor access, admin mark editing, server-side enforcement, and transcript/GPA persistence.

Generated from repository source on 2026-05-15.

Primary defense points

- Authentication is handled by Supabase Auth, but app roles come from public.app_users and are checked before role routes are shown.
- The login flow is protected by reCAPTCHA before AuthContext.login processes credentials.
- Advisor views are read-only and scoped to the logged-in advisor via advisorId.
- Changing marks is admin-only: route guard in App.tsx, UI validation in AdminStudentTranscriptPage, and token/admin verification in the server endpoint.
- Transcript marks are persisted in student_transcript_entries; SQL views recompute transcript rows, GPA, and completed credits for dashboards.

End-to-end trace

- 1. User opens /login, selects role, enters ID and password.
- 2. LoginPage obtains and verifies a reCAPTCHA token before credentials are processed.
- 3. AuthContext.login looks up the app user by university ID, checks selected role/status, and signs in with Supabase email/password.
- 4. AuthContext resolves the Supabase auth user back to public.app_users and stores the role-aware session.
- 5. App.tsx routes the user to the correct protected area. Admin transcript editing is only reachable through an admin route.
- 6. AppDataContext loads Supabase views into frontend state: courses, students, transcripts, GPA summaries, drafts, and evaluations.
- 7. Advisors inspect advisee transcript and risk views, guarded by advisorId === current user ID.
- 8. Admins open a student transcript, edit mark/attempt/term fields, and click Save changes.
- 9. The admin transcript page validates the edits and calls upsertTranscriptEntry for each changed row.
- 10. upsertTranscriptEntry revalidates, posts to /api/admin-upsert-transcript-entry, then refreshes derived frontend transcript/GPA state.
- 11. The server endpoint checks the bearer token belongs to an admin and writes student_transcript_entries using the service role.
- 12. SQL views expose updated transcript rows and recomputed GPA/completed credits back to the dashboards.

1. Login submission and security gate

Source: src/pages/LoginPage.tsx:61-88

- Most likely login question: what happens when the user presses sign in?
- This shows config checks, reCAPTCHA verification, AuthContext.login, and role-home navigation.

Code snippet

```
61   const handleSubmit = async (event: React.FormEvent) => {
62     event.preventDefault();
63     setError(null);
```

```

64
65   if (backendConfigError) {
66     setError(backendConfigError);
67     return;
68   }
69
70   if (!hasRecaptchaSiteKey()) {
71     setError('reCAPTCHA is not configured yet. Add the site key before logging in.');
```

2. AuthContext.login role and Supabase check

Source: src/context/AuthContext.tsx:689-748

- Most likely auth question: how do you prevent a user from selecting the wrong role?
- The app resolves the university ID, verifies role/status/email, signs in with Supabase, then confirms the auth user maps back to the same app user.

Code snippet

```

689   if (hasSupabaseConfig()) {
690     let matchedUser = usersRef.current.find((account) => account.id.toLowerCase() === normalizedId.toLowerCase());
691     const remoteMatchedUser = await fetchRemoteUserByUniversityId(normalizedId);
692     if (remoteMatchedUser) {
693       matchedUser = remoteMatchedUser;
694     }
695     if (!matchedUser || matchedUser.role !== role) {
696       return { success: false, error: registerFailedAttempt() };
697     }
698
699     if (matchedUser.status !== 'active') {
700       return { success: false, error: 'This account is inactive. Contact an administrator.' };
701     }
702
703     if (!matchedUser.email) {
704       return { success: false, error: 'This account is missing an email address in public.app_users. Add the email in Supabase and try again.' };
705     }
706
707     const signInResult = await supabaseSignInWithPassword(matchedUser.email, password);
708     if (signInResult.error) {
709       setIsAuthReady(true);
710       return { success: false, error: registerFailedAttempt() };
711     }
712
713     const resolved = await resolveAppUserFromAuth(signInResult.data.user ?? null);
```

```

714     if (!resolved || resolved.role !== role || resolved.id.toLowerCase() !== normalizedId.toLowerCase()) {
715         await supabaseSignOut();
716         setIsAuthReady(true);
717         return { success: false, error: 'Your authenticated Supabase account is not linked to the selected app user.' };
718     }
719
720     const lastLogin = new Date().toISOString();
721     setRememberSession(Boolean(rememberMe));
722     setUsers((current) =>
723         current.map((account) =>
724             account.id === resolved.id ? { ...account, lastLogin } : account
725         )
726     );
727     setUser({ ...resolved });
728     setAttempts((current) => {
729         const next = { ...current };
730         delete next[attemptKey];
731         return next;
732     });
733     setIsAuthReady(true);
734
735     void supabasePatch(
736         'app_users',
737         `university_id=eq.${encodeURIComponent(resolved.id)}`,
738         {
739             last_login_at: lastLogin,
740             status: matchedUser.status,
741             auth_user_id: resolved.authUserId,
742         }
743     ).catch((error) => {
744         console.error('Unable to update Supabase login metadata.', error);
745     });
746
747     void syncUsersFromSupabase();
748     return { success: true };

```

3. Protected routes and role boundaries

Source: src/App.tsx:25-90

- Most likely authorization question: how are student/advisor/admin screens separated?
- ProtectedRoute redirects unauthenticated users and blocks roles from routes they do not own.

Code snippet

```

25 function ProtectedRoute({
26     children,
27     allowedRoles,
28 }): {
29     children: ReactNode;
30     allowedRoles?: Role[];
31 } {
32     const { isAuthenticated, isAuthReady, user } = useAuth();
33
34     if (!isAuthReady) {
35         return <div className="min-h-screen bg-bg" />;
36     }
37
38     if (!isAuthenticated) {
39         return <Navigate to="/login" replace state={{ authError: 'Your session has expired or you must sign in to continue.' }} />;
40     }
41

```

```

42   if (allowedRoles && user && !allowedRoles.includes(user.role)) {
43     return <Navigate to={getHomeRoute(user.role)} replace />;
44   }
45
46   return <>{children}</>;
47 }
48
49 function RoleHomeRedirect() {
50   const { user } = useAuth();
51   return <Navigate to={user ? getHomeRoute(user.role) : '/login'} replace />;
52 }
53
54 export default function App() {
55   return (
56     <BrowserRouter>
57       <AuthProvider>
58         <AppDataProvider>
59           <MessagingProvider>
60             <Routes>
61               <Route path="/" element={<LandingPage />} />
62               <Route path="/login" element={<LoginPage />} />
63               <Route
64                 path="/app"
65                 element={
66                   <ProtectedRoute>
67                     <AppLayout />
68                   </ProtectedRoute>
69                 }
70             >
71               <Route index element={<RoleHomeRedirect />} />
72               <Route path="dashboard" element={<ProtectedRoute allowedRoles={['student']><StudentDashboard /></ProtectedRoute>} />
73               <Route path="courses" element={<ProtectedRoute allowedRoles={['student']><CoursePlanner /></ProtectedRoute>} />
74               <Route path="messages" element={<ProtectedRoute allowedRoles={['student']><StudentMessagesPage /></ProtectedRoute>} />
75               <Route path="profile" element={<Navigate to="/app/dashboard" replace />} />
76               <Route path="settings" element={<ProtectedRoute allowedRoles={['student']><StudentSettingsPage /></ProtectedRoute>} />
77               <Route path="advisor" element={<ProtectedRoute allowedRoles={['advisor']><AdvisorDashboard /></ProtectedRoute>} />
78               <Route path="advisor/student/:studentId" element={<ProtectedRoute allowedRoles={['advisor']><AdvisorStudentDetailPage /></ProtectedRoute>} />
79               <Route path="advisor/messages" element={<ProtectedRoute allowedRoles={['advisor']><AdvisorMessagesPage /></ProtectedRoute>} />
80               <Route path="advisor/courses" element={<Navigate to="/app/advisor" replace />} />
81               <Route path="advisor/reports" element={<Navigate to="/app/advisor" replace />} />
82               <Route path="advisor/settings" element={<ProtectedRoute allowedRoles={['advisor']><AdvisorSettingsPage /></ProtectedRoute>} />
83               <Route path="admin" element={<ProtectedRoute allowedRoles={['admin']><AdminDashboard /></ProtectedRoute>} />
84               <Route path="admin/students" element={<ProtectedRoute allowedRoles={['admin']><AdminStudentsPage /></ProtectedRoute>} />
85               <Route path="admin/students/:studentId/transcript" element={<ProtectedRoute allowedRoles={['admin']><AdminStudentTranscriptPage /></ProtectedRoute>} />
86               <Route path="admin/courses" element={<ProtectedRoute allowedRoles={['admin']><CourseManagement /></ProtectedRoute>} />
87               <Route path="admin/users" element={<ProtectedRoute allowedRoles={['admin']><UserManagementPage /></ProtectedRoute>} />
88               <Route path="admin/model" element={<Navigate to="/app/admin/courses" replace />} />
89               <Route path="admin/settings" element={<ProtectedRoute allowedRoles={['admin']><AdminSettingsPage /></ProtectedRoute>} />
90             </Routes>

```

4. Remote data snapshot after login

Source: src/context/AppDataContext.tsx:866-899

- Most likely data-flow question: where does dashboard/transcript data come from?
- This single snapshot loads courses, student summaries, transcript view rows, drafts, and related academic data from Supabase.

Code snippet

```

866 async function loadRemoteSnapshot(users: ReturnType<typeof useAuth>['users']) {

```

```

867   const [
868     academicTermRows,
869     departments,
870     settings,
871     courseRows,
872     prerequisiteRows,
873     corequisiteRows,
874     ruleRows,
875     statRows,
876     profileRows,
877     transcriptViewRows,
878     draftRows,
879     draftCourseRows,
880     evaluationRows,
881     passwordInquiryMessageRows,
882   ] = await Promise.all([
883     supabaseSelect<AcademicTermRow[]>('academic_terms', 'select=term_code,academic_year,term_name,term_type,max_credits'),
884     supabaseSelect<DepartmentRow[]>('departments', 'select=id,name'),
885     supabaseSelect<AppSettingRow[]>('app_settings', 'select=key,value_json'),
886     supabaseSelect<CourseRow[]>(
887       'courses',
888       'select=id,course_code,title,department_id,credits,course_type,is_plannable,internet_difficulty,difficulty_score,difficulty_basis,updated_at&order=course_code.asc
889     ),
890     supabaseSelect<CoursePrerequisiteRow[]>('course_prerequisites', 'select=course_id,prerequisite_course_id'),
891     supabaseSelect<CourseCorequisiteRow[]>('course_corequisites', 'select=course_id,corequisite_course_id'),
892     supabaseSelect<CourseRuleRow[]>('course_rules', 'select=course_id,rule_type,rule_value_int'),
893     supabaseSelect<HistoricalStatRow[]>(
894       'historical_course_stats',
895       'select=id,course_id,term_code,avg_grade,pass_rate,fail_rate,enrollment_count,withdrawals'
896     ),
897     supabaseSelect<StudentProfileRow[]>('student_dashboard_summary_v', 'select=student_id,student_name,department_name,advisor_id,gpa,admission_year,admission_term,completed_credits'),
898     supabaseSelect<StudentCompletedCourseRow[]>('student_transcript_v', 'select=id,student_id,term_code,term_type,course_code,course_name,credits,final_grade,status,attempt_no'),
899     supabaseSelect<ScheduleDraftRow[]>('schedule_drafts', 'select=id,student_id,name,term_code,status,saved_at&order=saved_at.desc'),

```

5. Advisor ownership guard

Source: src/pages/advisor/AdvisorStudentDetailPage.tsx:20-40

- Most likely advisor question: can an advisor see every student?
- The page only resolves a profile when the student advisorId matches the logged-in advisor ID.

Code snippet

```

20   export default function AdvisorStudentDetailPage() {
21     const { studentId = '' } = useParams();
22     const navigate = useNavigate();
23     const { user } = useAuth();
24     const {
25       courses,
26       getSelectedCourses,
27       getStudentDrafts,
28       getStudentTranscriptSemesters,
29       getStudentTermMetrics,
30       getStudentTranscript,
31       isAppDataReady,
32       plannerReviewSnapshots,
33       studentInsights,
34     } = useAppData();
35

```

```

36 const profile = studentInsights.find((item) => item.id === studentId && item.advisorId === user?.id) ?? null;
37 const selectedCourses = getSelectedCourses(studentId);
38 const drafts = getStudentDrafts(studentId);
39 const transcriptRows = getStudentTranscript(studentId);
40 const transcriptSemesters = getStudentTranscriptSemesters(studentId);

```

6. Admin transcript validation and save path

Source: `src/pages/admin/AdminStudentTranscriptPage.tsx:133-212`

- Most likely mark-changing question: how is input validated before saving?
- The editor accepts whole-number marks from 35 to 99, attempt numbers from 1 to 10, saves only changed rows, and sends each change through `upsertTranscriptEntry`.

Code snippet

```

133 const validationErrors = transcriptRows.flatMap((row) => {
134   const draft = getDraftForRow(row);
135   const finalGrade = parseGradeValue(draft.finalGrade);
136   const attemptNo = parseAttemptValue(draft.attemptNo);
137   if (draft.finalGrade.trim()) {
138     if (!/^^\d+$/.test(draft.finalGrade.trim())) {
139       return ['${row.courseCode} needs a whole-number mark, or a blank mark.'];
140     }
141
142     if (!Number.isInteger(finalGrade) || finalGrade === null || finalGrade < 35 || finalGrade > 99) {
143       return ['${row.courseCode} needs a whole-number mark from 35 to 99, or a blank mark.'];
144     }
145
146     if (!draft.attemptNo.trim()) {
147       return ['${row.courseCode} needs an attempt number from 1 to 10 when a mark is entered.'];
148     }
149   }
150
151   if (draft.attemptNo.trim()) {
152     if (!/^^\d+$/.test(draft.attemptNo.trim())) {
153       return ['${row.courseCode} needs a whole-number attempt from 1 to 10, or a blank attempt.'];
154     }
155
156     if (!Number.isInteger(attemptNo) || attemptNo === null || attemptNo < 1 || attemptNo > 10) {
157       return ['${row.courseCode} needs a whole-number attempt from 1 to 10, or a blank attempt.'];
158     }
159   }
160
161   return [];
162 });
163
164 const changedRows = transcriptRows.filter((row) => {
165   const draft = draftRows[getDraftKey(row)];
166   if (!draft) {
167     return false;
168   }
169
170   return !row.id
171     || draft.termCode !== row.termCode
172     || parseGradeValue(draft.finalGrade) !== row.finalGrade
173     || parseAttemptValue(draft.attemptNo) !== row.attemptNo;
174 });
175 const hasChanges = changedRows.length > 0;
176
177 const transcriptByTerm = transcriptRows.reduce<Record<string, StudentTranscriptRow[]>>((groups, row) => {

```

```

178     const draft = getDraftForRow(row);
179     const termCode = draft.termCode;
180     groups[termCode] = [...(groups[termCode] ?? []), row];
181     return groups;
182 }, {});
183
184 const handleReset = () => {
185     setDraftRows({});
186     setMessage(null);
187     setError(null);
188 };
189
190 const handleSave = async () => {
191     setMessage(null);
192     setError(null);
193
194     if (validationErrors.length > 0) {
195         setError(validationErrors[0]);
196         return;
197     }
198
199     setIsSaving(true);
200     for (const row of changedRows) {
201         const draft = draftRows[getDraftKey(row)];
202         const finalGrade = parseGradeValue(draft.finalGrade);
203         const attemptNo = parseAttemptValue(draft.attemptNo);
204         const result = await upsertTranscriptEntry({
205             id: row.id,
206             studentId: row.studentId,
207             termCode: draft.termCode || row.termCode || termOptions[0]?.termCode || '',
208             courseCode: row.courseCode,
209             finalGrade,
210             status: getStatusFromGrade(finalGrade),
211             attemptNo: attemptNo ?? Math.max(row.attemptNo, 1),
212         });

```

7. Admin transcript mark input UI

Source: `src/pages/admin/AdminStudentTranscriptPage.tsx:297-355`

- Most likely UI question: which exact fields are editable?
- The admin can change attempt number, semester taken, and mark; status is derived from the mark.

Code snippet

```

297         <td className="px-4 py-3 text-center">
298             <input
299                 type="number"
300                 min="1"
301                 max="10"
302                 step="1"
303                 inputMode="numeric"
304                 value={draft.attemptNo}
305                 onChange={(event) => setDraftRows((current) => ({
306                     ...current,
307                     [key]: { ...draft, attemptNo: event.target.value },
308                 })))
309                 className="w-20 rounded-lg border border-gray-200 px-3 py-2 text-center text-sm focus:border-[#2563eb] focus:outline-none focus:ring-2 focus:ring-[#2563eb]/30"
310             />
311         </td>
312     <td className="px-4 py-3">

```

```

313         <select
314             value={draft.termCode}
315             onChange={(event) => setDraftRows((current) => ({
316                 ...current,
317                 [key]: { ...draft, termCode: event.target.value },
318             })))
319             className="w-44 rounded-lg border border-gray-200 px-3 py-2 text-sm focus:border-[#2563eb] focus:outline-none focus:ring-2 focus:ring-[#
2563eb]/30"
320         >
321             {termOptions.map((term) => (
322                 <option key={term.termCode} value={term.termCode}>{term.termLabel}</option>
323             ))}
324         </select>
325     </td>
326     <td className="px-4 py-3">
327         <input
328             type="number"
329             min="35"
330             max="99"
331             step="1"
332             inputMode="numeric"
333             value={draft.finalGrade}
334             onChange={(event) => setDraftRows((current) => ({
335                 ...current,
336                 [key]: { ...draft, finalGrade: event.target.value },
337             })))
338             className="w-28 rounded-lg border border-gray-200 px-3 py-2 text-sm focus:border-[#2563eb] focus:outline-none focus:ring-2 focus:ring-[#
2563eb]/30"
339         />
340         <button
341             type="button"
342             onClick={() => setDraftRows((current) => ({
343                 ...current,
344                 [key]: { ...draft, finalGrade: '' },
345             })))
346             className="mt-2 text-xs font-semibold text-gray-500 transition-colors hover:text-[#2563eb]"
347         >
348             Clear grade
349         </button>
350     </td>
351     <td className="px-4 py-3 text-center">
352         <span className={`inline-flex rounded-full px-2.5 py-0.5 text-[10px] font-semibold ${getStatusClass(parsedGrade)}`}>
353             {getStatusLabel(parsedGrade)}
354         </span>
355     </td>

```

8. Client upsert: revalidate, call API, sync derived state

Source: `src/context/AppDataContext.tsx:1998-2054`

- Most likely state-management question: what happens after Save changes?
- The client normalizes status, validates again, calls the admin endpoint, then updates local transcript/GPA-derived state.

Code snippet

```

1998     const upsertTranscriptEntry = useCallback(async (input: TranscriptEntryInput): Promise<PlannerActionResult> => {
1999         const normalizedStatus = getTranscriptStatusForGrade(input.finalGrade, input.status);
2000         const normalizedInput = { ...input, status: normalizedStatus } satisfies TranscriptEntryInput;
2001         const validationError = validateTranscriptInput(normalizedInput, state.transcriptRows);
2002         if (validationError) {

```

```

2003     return { success: false, error: validationError };
2004 }
2005
2006 const course = state.courses.find((item) => item.code === normalizedInput.courseCode);
2007 if (!course) {
2008     return { success: false, error: 'Course was not found.' };
2009 }
2010
2011 let entryId = normalizedInput.id ?? createId();
2012
2013 if (hasSupabaseConfig()) {
2014     try {
2015         const result = await callAdminDataEndpoint<{ id?: string }>('/api/admin-upsert-transcript-entry', {
2016             id: entryId,
2017             existingEntry: Boolean(normalizedInput.id),
2018             studentId: normalizedInput.studentId,
2019             termCode: normalizedInput.termCode,
2020             courseCode: normalizedInput.courseCode,
2021             finalGrade: normalizedInput.finalGrade,
2022             status: normalizedStatus,
2023             attemptNo: normalizedInput.attemptNo,
2024         });
2025         entryId = result?.id ?? entryId;
2026     } catch (error) {
2027         return {
2028             success: false,
2029             error: error instanceof Error ? error.message : 'Unable to save transcript entry.',
2030         };
2031     }
2032 }
2033
2034 const nextRow = {
2035     id: entryId,
2036     studentId: normalizedInput.studentId,
2037     termCode: normalizedInput.termCode,
2038     termLabel: formatTermLabel(normalizedInput.termCode),
2039     termType: /summer/i.test(normalizedInput.termCode) ? 'summer' : 'regular',
2040     courseCode: course.code,
2041     courseName: course.name,
2042     credits: course.credits,
2043     finalGrade: normalizedInput.finalGrade,
2044     status: normalizedStatus,
2045     attemptNo: normalizedInput.attemptNo,
2046 } satisfies StudentTranscriptRow;
2047
2048 setState((current) => syncDerivedTranscriptState(current, [
2049     ...current.transcriptRows.filter((row) => row.id !== entryId),
2050     nextRow,
2051 ]));
2052
2053 return { success: true };
2054 }, [state.courses, state.transcriptRows, validateTranscriptInput]);

```

9. Server endpoint: admin-only service-role write

Source: `api/admin-upsert-transcript-entry.ts:45-65`

- Most likely security question: how do you stop non-admin users from changing marks?
- The API validates the bearer token with Supabase Auth and requires a matching `public.app_users` admin record before any write.

Code snippet

```
45 async function requireAdmin(request: VercelRequest, supabase: ReturnType<typeof getSupabaseAdminClient>) {
46   const token = String(request.headers.authorization ?? '').replace(/^Bearer\s+/i, '').trim();
47   if (!token) {
48     throw new Error('Missing admin session token.');
```

10. Server endpoint: validate and persist mark

Source: api/admin-upsert-transcript-entry.ts:76-193

- Most likely backend question: what is written to the database?
- The endpoint validates student/course/grade/attempt, resolves IDs, then updates an existing transcript row or upserts by student/course/term/attempt.

Code snippet

```
76 function validatePayload(payload: TranscriptEntryPayload) {
77   const studentId = String(payload.studentId ?? '').trim();
78   const termCode = String(payload.termCode ?? '').trim();
79   const courseCode = String(payload.courseCode ?? '').trim();
80   const finalGrade = payload.finalGrade ?? null;
81   const attemptNo = Number(payload.attemptNo);
82   const status = payload.status ?? 'in_progress';
83
84   if (!studentId || !termCode || !courseCode) {
85     return { error: 'Missing transcript entry fields.' };
86   }
87
88   if (!['passed', 'failed', 'withdrawn', 'in_progress'].includes(status)) {
89     return { error: 'Invalid transcript status.' };
90   }
91
92   if (finalGrade !== null && (!Number.isInteger(finalGrade) || finalGrade < 35 || finalGrade > 99)) {
93     return { error: 'Marks must be whole numbers from 35 to 99, or blank.' };
94   }
95
96   if (!Number.isInteger(attemptNo) || attemptNo < 1 || attemptNo > 10) {
97     return { error: 'Attempt number must be a whole number from 1 to 10.' };
98   }
99
100  return {
101    input: {
102      id: payload.id ? String(payload.id) : undefined,
103      existingEntry: payload.existingEntry === true,
```

```

104     studentId,
105     termCode,
106     courseCode,
107     finalGrade,
108     status: getStatusForGrade(finalGrade),
109     attemptNo,
110   },
111 };
112 }
113
114 export default async function handler(request: VercelRequest, response: VercelResponse) {
115   if (request.method !== 'POST') {
116     response.status(405).json({ success: false, error: 'Method not allowed.' });
117     return;
118   }
119
120   try {
121     const payload = typeof request.body === 'string'
122       ? JSON.parse(request.body) as TranscriptEntryPayload
123       : request.body as TranscriptEntryPayload;
124     const validation = validatePayload(payload);
125     if ('error' in validation) {
126       response.status(400).json({ success: false, error: validation.error });
127       return;
128     }
129
130     const supabase = getSupabaseAdminClient();
131     await requireAdmin(request, supabase);
132
133     const { input } = validation;
134     const { data: student, error: studentError } = await supabase
135       .from('app_users')
136       .select('id')
137       .eq('university_id', input.studentId)
138       .eq('role', 'student')
139       .maybeSingle();
140
141     if (studentError) {
142       throw studentError;
143     }
144
145     if (!student) {
146       response.status(404).json({ success: false, error: 'Student account was not found.' });
147       return;
148     }
149
150     const { data: course, error: courseError } = await supabase
151       .from('courses')
152       .select('id')
153       .eq('course_code', input.courseCode)
154       .maybeSingle();
155
156     if (courseError) {
157       throw courseError;
158     }
159
160     if (!course) {
161       response.status(404).json({ success: false, error: 'Course was not found.' });
162       return;
163     }

```

```

164
165     const transcriptPayload = {
166       id: input.id,
167       student_id: student.id,
168       term_code: input.termCode,
169       course_id: course.id,
170       final_grade: input.finalGrade,
171       status: input.status,
172       attempt_no: input.attemptNo,
173     };
174
175     const query = input.existingEntry && input.id
176       ? supabase
177         .from('student_transcript_entries')
178         .update(transcriptPayload)
179         .eq('id', input.id)
180         .select('id')
181         .single()
182       : supabase
183         .from('student_transcript_entries')
184         .upsert(transcriptPayload, { onConflict: 'student_id,course_id,term_code,attempt_no' })
185         .select('id')
186         .single();
187
188     const { data: savedEntry, error: saveError } = await query;
189     if (saveError) {
190       throw saveError;
191     }
192
193     response.status(200).json({ success: true, id: savedEntry.id });

```

11. Database transcript table

Source: supabase/007_transcript_first_academic_schema.sql:29-40

- Most likely schema question: where are marks stored?
- student_transcript_entries is the canonical table for term, course, final_grade, status, and attempt number.

Code snippet

```

29 create table if not exists public.student_transcript_entries (
30   id uuid primary key default gen_random_uuid(),
31   student_id uuid not null references public.app_users(id) on delete cascade,
32   term_code text not null,
33   course_id uuid not null references public.courses(id) on delete restrict,
34   final_grade numeric(5,2),
35   status text not null check (status in ('passed', 'failed', 'withdrawn', 'in_progress')),
36   attempt_no integer not null default 1 check (attempt_no >= 1),
37   created_at timestampz not null default timezone('utc', now()),
38   updated_at timestampz not null default timezone('utc', now()),
39   unique (student_id, course_id, term_code, attempt_no)
40 );

```

12. Database views derive transcript, GPA, and credits

Source: supabase/007_transcript_first_academic_schema.sql:215-323

- Most likely dashboard question: how does changing a mark affect GPA?
- Views expose joined transcript rows and recompute GPA/completed credits from the transcript table.

Code snippet

```

215 create or replace view public.student_transcript_v as

```

```

216 select
217     ste.id,
218     ste.student_id,
219     ste.term_code,
220     coalesce(at.term_type, public.term_type_from_term_name(public.term_name_from_code(ste.term_code))) as term_type,
221     c.course_code,
222     c.title as course_name,
223     c.credits,
224     ste.final_grade,
225     ste.status,
226     ste.attempt_no
227 from public.student_transcript_entries ste
228 join public.courses c on c.id = ste.course_id
229 left join public.academic_terms at on at.term_code = ste.term_code;
230
231 create or replace view public.student_term_metrics_v as
232 select
233     ste.student_id,
234     ste.term_code,
235     coalesce(at.term_type, public.term_type_from_term_name(public.term_name_from_code(ste.term_code))) as term_type,
236     count(*)::integer as course_count,
237     coalesce(sum(case when ste.status = 'passed' then c.credits else 0 end), 0)::integer as completed_credits,
238     round((avg(public.normalize_transcript_mark(ste.final_grade)) * 4 / 100.0)::numeric, 2) as gpa
239 from public.student_transcript_entries ste
240 join public.courses c on c.id = ste.course_id
241 left join public.academic_terms at on at.term_code = ste.term_code
242 group by
243     ste.student_id,
244     ste.term_code,
245     coalesce(at.term_type, public.term_type_from_term_name(public.term_name_from_code(ste.term_code)));
246
247 create or replace view public.student_dashboard_summary_v as
248 with transcript_marks as (
249     select
250         ste.student_id,
251         round((avg(public.normalize_transcript_mark(ste.final_grade)) * 4 / 100.0)::numeric, 2) as gpa
252     from public.student_transcript_entries ste
253     where ste.final_grade is not null
254     group by ste.student_id
255 ),
256 completed_credit_totals as (
257     select
258         passed.student_id,
259         coalesce(sum(c.credits), 0)::integer as completed_credits
260     from (
261         select distinct student_id, course_id
262         from public.student_transcript_entries
263         where status = 'passed'
264     ) passed
265     join public.courses c on c.id = passed.course_id
266     group by passed.student_id
267 ),
268 latest_draft as (
269     select distinct on (student_id)
270         id,
271         student_id,
272         name,
273         term_code,
274         status,
275         saved_at

```

```

276     from public.schedule_drafts
277     order by student_id, saved_at desc
278 ),
279 latest_evaluation as (
280     select distinct on (student_id)
281         id,
282         schedule_id,
283         student_id,
284         total_score,
285         risk_label,
286         total_credits,
287         recommendations,
288         explanation,
289         evaluated_at
290     from public.schedule_evaluations
291     order by student_id, evaluated_at desc
292 )
293 select
294     student_user.id as student_id,
295     student_user.university_id,
296     student_user.full_name as student_name,
297     coalesce(transcript_marks.gpa, 0) as gpa,
298     coalesce(completed_credit_totals.completed_credits, 0) as completed_credits,
299     sp.admission_year,
300     sp.admission_term,
301     d.name as department_name,
302     advisor_user.id as advisor_id,
303     advisor_user.full_name as advisor_name,
304     latest_draft.id as latest_draft_id,
305     latest_draft.name as latest_draft_name,
306     latest_draft.term_code as latest_draft_term_code,
307     latest_draft.status as latest_draft_status,
308     latest_draft.saved_at as latest_draft_saved_at,
309     latest_evaluation.id as latest_evaluation_id,
310     latest_evaluation.total_score as latest_total_score,
311     latest_evaluation.risk_label as latest_risk_label,
312     latest_evaluation.total_credits as latest_total_credits,
313     latest_evaluation.recommendations as latest_recommendations,
314     latest_evaluation.explanation as latest_explanation,
315     latest_evaluation.evaluated_at as latest_evaluated_at
316 from public.student_profiles sp
317 join public.app_users student_user on student_user.id = sp.user_id
318 join public.departments d on d.id = sp.department_id
319 left join public.app_users advisor_user on advisor_user.id = sp.advisor_id
320 left join transcript_marks on transcript_marks.student_id = sp.user_id
321 left join completed_credit_totals on completed_credit_totals.student_id = sp.user_id
322 left join latest_draft on latest_draft.student_id = sp.user_id
323 left join latest_evaluation on latest_evaluation.student_id = sp.user_id;

```